

1 Lesson 22-09-26

1.1 Introduction

The course aims to give an introduction to the use of **HDLs** (*Hardware Description Languages*):

- We will see HDLs used as *specification languages* used to design hardware. Both *Verilog* and *VHDL* are used, respectively industry and academic based;
- High level concepts will be studied at the **RT** (*Register Transfer*) level of abstraction. We'll also see design methodologies for **ASICs** (*Application-Specific Integrated Circuits*) and **FPGAs** (*Field Programmable Gate Arrays*);
- Simulation tools are the main tool available for verification and validation of designs. Other important tools include **CAD**-based formal verification, and mathematical *formal verification* methods.

The topics covered will be:

- Usage of **simulation** tools such as the Xilinx Vivado suite of programs;
- **Combinatorial** networks, that is networks where the output value is only determined by the values taken by their inputs;
- **Sequential** networks, that is networks where the output value is determined both by the values taken by the inputs and internal state. Sequential networks can either be *asynchronous* or *synchronized* to an external clock signal;
- **FSMs** (*Finite State Machines*), which we will see as a specific class of sequential circuits;
- **FSM-D** models (FSMs operating at logical level, controlling *datapaths* for data manipulation);
- **RAMs** (*Random Access Memories*) and **ROMs** (*Read-Only Memories*), and their respective interfaces facing other components (including ones we will design ourselves).

1.1.1 VHDL as a specification language

VHDL will be used and explained in a design-oriented approach covering:

1. **Combinatorial** logic;
2. *Synchronous* **sequential** logic (with *asynchronous* sequential logic being mostly a less useful superset);
3. **Datapath** components.

Verilog will be covered, by *difference* from VHDL. Still, both languages are useful in academic and industrial contexts, and proficiency in both is expected in most design environments.

1.1.2 Register Transfer level designs

We will then move upwards to the **RT** (*Register Transfer*) level of abstraction. RT level is a design abstraction which models a synchronous digital circuit in terms of the flow of digital signals (data) between hardware registers, and the logical operations performed on those signals.

Using this abstraction we will design embedded systems, covering design *metrics* (related to economical, manufacturing, ... factors), design *methodologies* for **single-purpose** processors.

By *single-purpose*, we mean processors that implement a specific algorithm in hardware (ASICs, FPGAs): this is the opposite concept to *general-purpose* computing.

FSM (*Finite State Machine*) optimization will also be covered.

1.1.3 Formal verification

The main tool used to verify a design is **simulation**, and simulation through the Xilinx suite of programs will be studied.

Another method that will be studied is **formal verification**, that is mathematical methods used to prove the validity of designs independent of the given inputs:

- *Theorem proving* involves expressing the hardware and desired property mathematically, then proving the property from axioms/rules;
- *Equivalence checking* entails, given two designs, proving that they behave equivalently under specified conditions;
- *Model checking* means determining if certain *properties* of our designed system can be proved to be true independently of the inputs.

1.2 VHDL for digital design

A **digital system** is an electronic system operating on data encoded as a sequence of zeroes and ones (low and high, off or on values).

When digital design started (around the 1960s), the number of components in a circuit was very limited. As of today, that number has grown up to billions of components on a single chip.

This growth has made the *documentation* process harder and harder, and the resulting artifacts more and more imprecise and ambiguous.

1.2.1 HDLs

The main problem is that *natural language* is not fit for description of a digital system. For this reason, machine-readable textual languages were invented for describing hardware. We call these languages **HDLs** (*Hardware Description Languages*).

HDLs are the opposite of *programming* languages: they don't describe *how* an algorithm is implemented, can't be executed, but describe the *temporal behavior* and *spatial structure* of an hardware system. This allows to combine design and documentation in a single source of truth (documentation of a system can be extracted from its HDL description).

An HDL usually allows us to define:

- A **black box** representation of a given system, which only entails its inputs and outputs (which we call the *interface* of the system);

- An **architectural** view of the system, which entails how the inputs (and optionally, previous state) are combined to generate the outputs. The architecture of a system can be defined in terms of other systems (such as *And*, *Or* and *Not* gates), themselves taken as other black boxes with their own architectural implementation.

Of course, the set of components used to make up the architectural description of a system represents its *spatial structure*. The *temporal behavior*, on the other hand, is specified using other language constructs that specify the timing constants between components.

1.2.2 VHDL

VHDL (*VHSIC Hardware Description Language*), with **VHSIC** (*Very High Speed Integrated Circuit*) being a project of the U.S. Dept. of Defense, is a hardware specification language based on a top-bottom approach. The syntax is similar to *Ada*, another U.S. Dept. of Defense creation. Today it's mainly used in academia, and is better suited to high-level behavioral modeling.

1.2.3 Verilog

Verilog was defined in 1995 at *Gateway Design Automation Inc.*, then acquired by *Cadence Design Systems*. The initial implementation was proprietary, and meant for designers to quickly iterate over prototypes in a bottom-up approach. Afterwards, the language was extended and made into an open standard. The syntax is C-like, and today it's the most significant difference from VHDL. Another difference is that Verilog is more fit for low-level work than VHDL.

1.2.4 Comparison of VHDL and Verilog

We said that Verilog will be studied by difference.

Other differences to note are noted in the table:

VHDL	Verilog
Ada-like, not <i>case sensitive</i>	C-like, <i>case sensitive</i>
Strongly typed, compile time error checking	Weakly typed, less compile time checking, more likely runtime bugs
Focused on safety	Focused on writing models quickly
User-defined types, 9 value logic	No user-defined types, 4 value logic
Hierarchy through entity definitions and implementation through architecture	Hierarchy through module definitions with self contained implementation
Gate level modeling with FPGA library support	Better ASIC support, user defined primitives
Assertions	No assertions
No named events	Named events
Partial interface abstraction via component definitions	No interface abstraction
Packaging via actual packages	Packaging via include files
Useful for high-level modeling	Better suited for low-level modeling

More verbose, meant to act both as design and documentation	Less verbose, no packaging capability, harder to extend
Unambiguous semantics and easily portable designs, no race conditions	Ambiguous emulation semantics, race conditions possible, different vendors might have different semantics
No simulation control defined in language	Basic simulation control defined in language

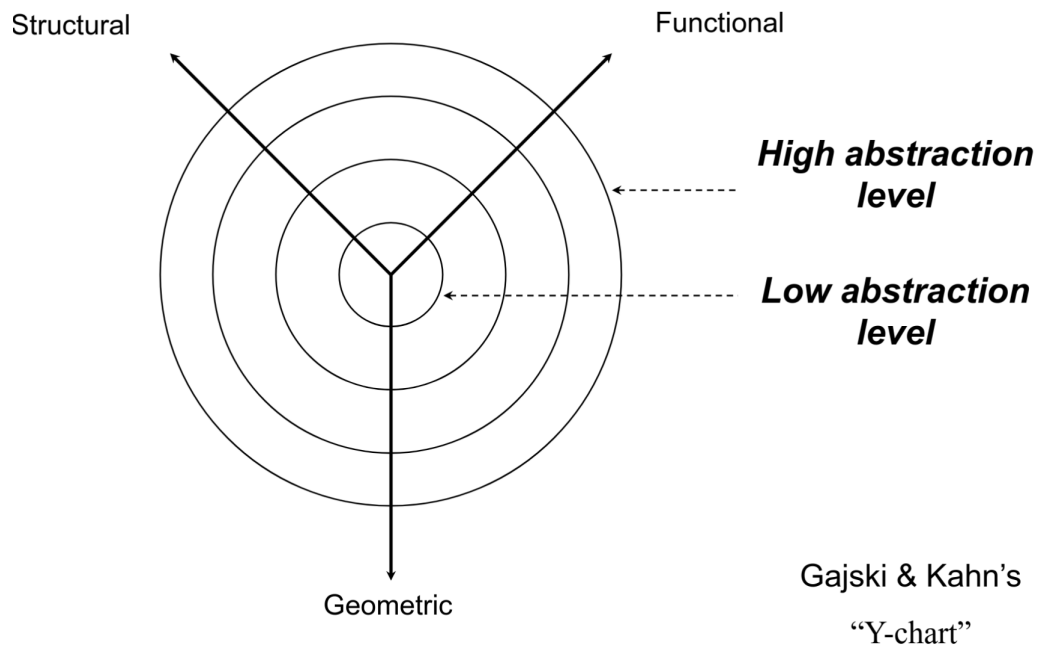
1.2.5 Synthesis

Today, HDLs are increasingly used for designing ICs in a *top-down* approach involving **synthesis**:

1. We *design* the circuit through high-level descriptions;
2. We leave the conversion to a low-level representation to *synthesis* tools, which automatically convert HDL behavior to HDL circuits;
3. The last step is, of course, *simulation* to validate the design against the original high-level description.

Synthesis is not the objective of this course. On the other hand, the use of HDLs for synthesis is growing as circuits get more complex, and synthesis tools are maturing. One thing to note is that HDLs were originally defined for simulation: some features of the language are not supported by synthesizers (for instance, timing constants don't necessarily translate to low-level components, neither do branching loops such as the `while` loop offered by HDLs).

1.2.6 The Y-chart



There are different ways of modeling a given system, each with different levels of abstraction:

- **Functional:** what the system *does*: this goes all the way from algorithmic (behavioral) representation to RT languages, boolean functions, up to the *differential equations* which govern the electronic components that make up the system;
- **Structural:** what the system is *made up of*: we can start from high-level blocks such as *processors* and *memories*, look at components which operate at RT level, up to single logic gates and their implementations based on transistors;
- **Geometric:** how the components of the system are *laid out* physically. This won't be of much interest in this course, as the focus is on *logical* modeling.

This relates to a set of ways we can model our systems:

- At the *system* level, we see systems as a set of interconnected high-level components and their interfaces (black boxes);
- At an *algorithmic* level, we see systems through the algorithms they implement;
- At the *RT* level, we see systems through registers and sequential networks that propagate values across them;
- At the *logic* level, we see systems through combinatorial networks (made up of gates) implementing boolean functions;
- At the *transistor* level, we see how gates themselves are implemented through transistors;
- At the *geometry/topology* level, we see how physical components (transistors) are laid out on boards.

The data type we use today in VHDL modeling is the `std_logic` type, which implements 9-value logic (compared to other versions, which use simple 2-value logic `bit` values).